01_m0_m2

M0 - Foundation, toolchain & dbt config [Wk1-MVP]

The very first move is **not** writing code. It is making Claude Code's memory correct. `CLAUDE.md` currently sits in `docs/`, so it does **not** auto-load at the start of a session and Claude has no canonical-name anchor (D1 violated by default). M0's first action moves it to the repo root, then scaffolds the dbt project skeleton by *copying* the configs that already exist verbatim in `DBT_GUIDE.md` sec 1 - Claude wires, it does not design.

Context to load

- Open `DBT_GUIDE.md` **sec 1** (the project tree + `dbt_project.yml` / `packages.yml` / `profiles.yml` + Conventions) - the source of truth for every config file.
- Open `MCP_ARCHITECTURE.md` **sec 3** (the `mcp_servers.yaml` block) only for the stub.
- Confirm `CLAUDE.md` will auto-load: after the move, it must be at `helios/CLAUDE.md` (repo root). Until then, **attach it manually** in the M0 session so the canonical vocabulary (session_key, reached_*, 10 channel groups, layer prefixes) is in context.
- Reality check before scaffolding: the repo today contains only `docs/`, `models/semantic/` (the M5 `semantic_layer.yaml` already exists), `eval/`, and `PROJECT_STRUCTURE.md`. No `dbt_project.yml` yet. Do not let Claude assume a dbt project is already initialised.

Files to provide

- `@docs/DBT_GUIDE.md` (sec 1) - the literal text of `dbt_project.yml`, `packages.yml`, `profiles.yml`.
- `@docs/CLAUDE.md` (the file being moved) - so Claude knows the conventions and the target layout in sec 6.
- `@docs/MCP_ARCHITECTURE.md` (sec 3) - for the `mcp_servers.yaml` stub.
- Nothing else. M0 touches no SQL and no GA4 columns, so do not attach the data-model docs.

Prompts to use

First, the CLAUDE.md move (a one-liner, but do it as an explicit step so the rest of M0 benefits):

```
[ROLE] You are setting up the Helios repo for milestone M0. CLAUDE.md is the auto-load
memory file but it currently lives in docs/, so it does NOT auto-load.
[TASK] Move docs/CLAUDE.md to the repo root (helios/CLAUDE.md). Update the one self-
reference line in it ("CLAUDE.md  ← this file" already shows it at root, so the file body
is correct - only the path changes). Then update docs/DEPENDENCY_MAP.md and any doc that
links to docs/CLAUDE.md to point at ./CLAUDE.md.
[CONSTRAINTS] Do not edit the body of CLAUDE.md other than fixing stale relative links.
Show me `git mv` and the diff. Do not create a copy - move it.
```

Then enter **plan mode** for the scaffold (Shift+Tab) and use:

```
[ROLE] You are implementing Helios milestone M0 (repo scaffold + dbt config). Follow
CLAUDE.md conventions (now auto-loaded at repo root).
[SOURCE OF TRUTH] Copy/adapt the four config files VERBATIM from DBT_GUIDE.md sec 1
(@docs/DBT_GUIDE.md): dbt_project.yml, packages.yml, profiles.yml. Add a .gitignore,
requirements.txt, and the mcp_servers.yaml STUB from MCP_ARCHITECTURE.md sec 3
(@docs/MCP_ARCHITECTURE.md) at the repo root.
[TASK] In plan mode, propose:
  1. dbt_project.yml at repo root - copy sec 1 exactly: vars (ga4_start_date 2020-11-01,
     ga4_end_date 2021-01-31, incremental_lookback_days 3, engagement_time_threshold_msec
     10000), the per-layer +configs (staging=view+access:private, intermediate=ephemeral,
     marts core=incremental insert_overwrite partitioned by event_date clustered by
     device_category,channel_group, dims=table). Set profile: helios.
  2. packages.yml - dbt_utils, dbt_expectations, dbt_project_evaluator, codegen with the
     exact version pins from sec 1.
  3. profiles.yml - helios profile, dev target oauth/helios_dev/location US/
     maximum_bytes_billed 5368709120; prod target as in sec 1.
  4. requirements.txt - dbt-bigquery, plus the Python deps the project will need (do NOT
     invent versions; pin dbt-bigquery≥1.7).
  5. .gitignore - target/, dbt_packages/, logs/, *.json keyfiles, .env, __pycache__/.
  6. mcp_servers.yaml STUB at repo root - copy the servers block from MCP_ARCHITECTURE.md
     sec 3; registry must point at ./models/semantic/semantic_models.yml.
  7. Create the empty model folder skeleton from CLAUDE.md sec 6 (models/staging,
     models/intermediate, models/marts/{core,finance,growth}, macros/, seeds/, tests/).
[CONSTRAINTS] Use ONLY the values present in the attached doc sections. Do NOT invent a
GCP project name other than helios-analytics (the value in sec 1). If a value is missing,
STOP and ASK - do not guess credentials, dataset names, or versions.
[OUTPUT] Show me the plan first. For each file, cite the DBT_GUIDE.md sec 1 sub-block it
came from. After I approve, write the files, then run `dbt deps` and `dbt debug` and show
the full output.
```

Preventing hallucinations

- D1 in action: the **whole point of M0** is to fix auto-load. Verify `CLAUDE.md` is at the root with `git ls-files CLAUDE.md` (must return `CLAUDE.md`, not `docs/CLAUDE.md`); a fresh session afterward should show it loaded.
- D2/D5: the configs are *transcribed*, not authored. The biggest hallucination risk is Claude "improving" values - inventing a `maximum_bytes_billed`, a different `location`, a

`priority`, or a 6th MCP server. Pin them: `maximum_bytes_billed: 5368709120` (5 GiB), `location: US`, `project: helios-analytics`, exactly the 5 servers from MCP sec 3. Grep the result: `Select-String "5368709120|location: US|helios-analytics" dbt_project.yml,profiles.yml`.

- The mcp_servers.yaml is a **stub** - Claude must not flesh out tool implementations in M0 (those are M6). The registry path must be `./models/semantic/semantic_models.yml` and that file already exists, so a typo'd path is catchable.
- D6 - let the machine catch it: `dbt debug` is the M0 acceptance gate. It validates `profiles.yml` parses, the BigQuery connection authenticates via ADC, `dbt_project.yml` is well-formed, and packages resolve. If Claude hallucinated a malformed `+config` key, `dbt parse` / `dbt debug` fails immediately.

Reviewing generated code

- R2/R3: run and show `dbt deps` (packages install clean) then `dbt debug` - **every check must be green**: profiles.yml found, dbt_project.yml valid, connection ok. Do not accept M0 without a green `dbt debug`.
- R5 (read the diff against conventions): confirm staging/intermediate are `+access: private`; marts core is `incremental` / `insert_overwrite` /partition `event_date` /cluster `device_category, channel_group` / `require_partition_filter: true`; the four dims override back to `table` + `require_partition_filter: false`. These are the load-bearing cost/governance controls from sec 1.
- Confirm `vars` are present and exact (the build window and the 10000 ms engagement threshold) - downstream macros read `var('engagement_time_threshold_msec')`, so a missing var breaks M1/M3.
- Confirm `.gitignore` excludes `target/`, `dbt_packages/`, and any `*.json` keyfile (no service-account key should ever be committed - dev uses OAuth/ADC per sec 1).
- Confirm `CLAUDE.md` moved (not copied) and no dangling `docs/CLAUDE.md` link remains.
- /clear before M1.

M1 - Macros, sources & seed [Wk1-MVP]

M1 builds the shared abstractions that every downstream model depends on: the **four macros** (`get_event_param`, `sessionize`, `channel_group` / `channel_group_case`, and the custom generic test `test_revenue_reconciles`), the `src_ga4` source declaration, and the `channel_group_mapping.csv` seed. All of this exists verbatim in `DBT_GUIDE.md` sec 3.4 (macros), sec 2 / sec 3.1 (source), and sec 6.4 (the test). Claude copies the macro bodies exactly - the

`channel_group_case()` regex precedence and the `sessionize()` MD5 concat order are canonical and must not be paraphrased.

Context to load

- `DBT_GUIDE.md` **sec 3.4** - the three macro bodies (`get_event_param`, `sessionize`, `channel_group_case`) verbatim.
- `DBT_GUIDE.md` **sec 2 + sec 3.1** - the `src_ga4` source declaration (`_ga4__sources.yml` / `src_ga4.yml`): `database: bigquery-public-data`, `schema: ga4_obfuscated_sample_ecommerce`, `identifier: 'events_*'`, the freshness contract (production-real, sample-informational).
- `DBT_GUIDE.md` **sec 6.4** - the `test_revenue_reconciles` custom generic test body.
- `CLAUDE.md` (auto-loaded) sec 4-5 - the 10 channel groups, the canonical `session_key` expression, the `traffic_source` gotcha, money/`_in_usd` rule.

Files to provide

- `@docs/DBT_GUIDE.md` (secs 2, 3.1, 3.4, 6.4) - the literal macro/source/test text.
- The **GA4 source schema** so the source declaration references real raw columns: attach the column list from `DBT_GUIDE.md` sec 2's `_ga4__sources.yml` (event_date, event_timestamp, event_name, user_pseudo_id, ecommerce.transaction_id) - and note the GA4 export schema is public (`bigquery-public-data.ga4_obfuscated_sample_ecommerce`), so the macros may only reference fields that exist there (`event_params[].key`, `event_params[].value.{string,int,float,double}_value`, `traffic_source.{source,medium,name}`, `device.*`, `geo.*`, `ecommerce.*`).
- `@models/semantic/semantic_layer.yaml` - the registry exists already; `channel_group` is a canonical dimension there. Keep it open so the seed's `channel_group` values match the 10 governed names exactly.

Prompts to use

Set up the slash commands first (cheap, paid off all week). Create `.claude/commands/new-model.md` and `.claude/commands/new-metric.md`:

```
[TASK] Create two custom slash commands for this repo.
1. .claude/commands/new-model.md - "/new-model <layer> <name>": scaffold a dbt model file
   at models/<layer>/<name>.sql + a stub entry in that folder's _*__models.yml, using the
   conventions in CLAUDE.md (layer prefix stg_/int_/fct_/dim_, materialization per
   dbt_project.yml, snake_case). The command body must instruct: attach the relevant
   DBT_GUIDE.md section, reference only real upstream columns, write the schema test
   (unique+not_null on the grain key) BEFORE the SQL, then run `dbt parse`.
2. .claude/commands/new-metric.md - "/new-metric <name>": add a metric to
```

```
   models/semantic/semantic_layer.yaml (NOT hand-written SQL), add its test, and re-run
   validate_semantic.py. The body must forbid inventing column names not in the registry.
Use $ARGUMENTS in each command. Show me both files.
```

The macros (one focused session, plan mode):

```
[ROLE] You are implementing Helios M1, files macros/get_event_param.sql,
macros/sessionize.sql, macros/channel_group.sql, and tests/generic/
test_revenue_reconciles.sql. Follow CLAUDE.md conventions (auto-loaded).
[SOURCE OF TRUTH] Copy the macro bodies VERBATIM from DBT_GUIDE.md sec 3.4
(@docs/DBT_GUIDE.md) and the test from sec 6.4. Do not rewrite or "clean up" the regexes
or the MD5 concat.
[UPSTREAM] These macros run against the raw GA4 export. Reference ONLY columns that exist
in bigquery-public-data.ga4_obfuscated_sample_ecommerce: event_params (ARRAY<STRUCT key,
value>), traffic_source.{source,medium,name}, device.*, geo.*, ecommerce.* per the
attached sec 2 column list.
[TASK]
  - get_event_param(key, type='string'): the typed correlated-subquery extractor (string/
    int/float/double value slots) exactly as sec 3.4.
  - sessionize(): TO_HEX(MD5(CONCAT(user_pseudo_id,'-',CAST(ga_session_id AS STRING)))) -
    the ONE canonical session_key expression. Never FARM_FINGERPRINT.
  - channel_group()/channel_group_case(): the 10-group CASE (Direct, Organic Search, Paid
    Search, Display, Paid Social, Organic Social, Email, Affiliates, Referral, Other),
    gclid-aware, in the exact precedence from sec 3.4. NO 11th group, no "Paid Other".
  - test_revenue_reconciles: the custom generic test from sec 6.4 (0.5% tolerance vs raw
    purchase_revenue_in_usd where event_name='purchase').
[CONSTRAINTS] Use ONLY canonical names from CLAUDE.md / semantic_layer.yaml. If you need a
column not in the attached GA4 schema, STOP and ASK.
[TEST-FIRST] Then run `dbt parse` (must compile the macros with no Jinja/ref errors).
[OUTPUT] Cite the sec 3.4 / 6.4 sub-block for each macro.
```

The source + seed:

```
[ROLE] Helios M1, files models/staging/_ga4__sources.yml and
seeds/channel_group_mapping.csv. Follow CLAUDE.md conventions.
[SOURCE OF TRUTH] Copy the src_ga4 declaration from DBT_GUIDE.md sec 2/3.1
(@docs/DBT_GUIDE.md): name src_ga4, database bigquery-public-data, schema
ga4_obfuscated_sample_ecommerce, table events identifier 'events_*', the freshness
block + loaded_at_field, and the not_null source-column tests.
[TASK] Write _ga4__sources.yml exactly as the doc. Then create the seed
channel_group_mapping.csv (columns: source, medium, channel_group) whose channel_group
values are ONLY the 10 canonical groups - cross-check against channel_group_case() and
semantic_layer.yaml (@models/semantic/semantic_layer.yaml).
[CONSTRAINTS] The 10 channel_group values must match the macro exactly; no 11th value.
[TEST-FIRST] Run `dbt parse` then `dbt seed --select channel_group_mapping` and show output.
[OUTPUT] Cite sec 2/3.1.
```

Preventing hallucinations

- D3/D5 (real upstream columns): the single biggest M1 risk is the macros referencing a GA4 field that doesn't exist or has the wrong path. Pin: `event_params` is `ARRAY<STRUCT<key STRING, value STRUCT<string_value, int_value, float_value, double_value, ... >>>` ; session id lives in `event_params.ga_session_id` (an int param), **not** a top-level column; source/medium are session-scoped `event_params.source/medium` , with `traffic_source.*` as first-touch fallback only (the gotcha in CLAUDE.md sec 5). If Claude reaches for `traffic_source` as the primary source, that is the canonical bug - reject it.
- D4 (cite): require Claude to cite sec 3.4 for each macro. The `channel_group_case()` regex precedence (Direct -> Paid Search -> Paid Social -> Display -> Organic Search -> ...) is order-sensitive; a reordered CASE produces silently wrong attribution. Diff the regexes against sec 3.4 character-for-character.
- D5 (no invention): the seed's `channel_group` column must contain exactly the 10 canonical strings. Run `Get-Content seeds/channel_group_mapping.csv | Select-Object -Skip 1 | ForEach-Object { ($_ -split ',')[2] } | Sort-Object -Unique` and confirm the set is the 10 names with no extras (no "Paid Other", no 11th).
- D6 (machine catch): `dbt parse` compiles the macros and the source YAML; a Jinja typo or a malformed `source()` declaration fails here before any model uses it. There are no models yet, so `dbt parse` is the cheap, fast M1 gate.

Reviewing generated code

- R2: `dbt parse` clean (macros compile, source YAML valid). `dbt seed --select channel_group_mapping` loads without a type error (the seed `+column_types` from M0's dbt_project.yml force source/medium/channel_group to string).
- R4 (canonical names): grep the macro file - `sessionize` must contain `to_hex(md5(concat(` and `user_pseudo_id` / `ga_session_id` , and must **not** contain `farm_fingerprint` . `channel_group_case` must contain all 10 group literals and no others.
- R5: confirm `get_event_param` selects the right value slot per type (string->string_value, int->int_value, etc.) and uses `where ep.key = '{{ key }}'` with `limit 1` - a wrong slot silently returns null for every int param (ga_session_id, engagement_time_msec), which would zero out sessionization downstream.
- R1: each macro block cites its sec 3.4 origin; the test cites sec 6.4.
- The `test_revenue_reconciles` body: confirm it filters `event_name = 'purchase'` on the raw side and uses the 0.5% relative-drift tolerance - it can't be exercised until a revenue mart exists (M4), but it must `dbt parse` now.
- /clear before M2.

M2 - Staging models [Wk1-MVP]

M2 builds the two staging views - `stg_ga4__events.sql` (1:1 typed/renamed events, `session_key` via `sessionize()`) and `stg_ga4__event_params.sql` (the long unnested param table) - plus `stg_ga4__schema.yml`. Both are verbatim in `DBT_GUIDE.md` sec 3.2, 3.3, 3.5. Staging is *pure projection*: **no joins, no aggregations, no `SELECT *` against source, no business logic** (sec 3 preamble). This is the first milestone that produces real data, so it's the first chance for hallucinated GA4 columns to leak - the source schema must be attached.

Context to load

- `DBT_GUIDE.md` **sec 3 preamble** (the staging discipline rules) + **sec 3.2** (`stg_ga4__events`) + **sec 3.3** (`stg_ga4__event_params`) + **sec 3.5** (`stg_ga4__schema.yml`).
- The **M1 macros** - staging *calls* `get_event_param()` and `sessionize()`. Keep `macros/get_event_param.sql` and `macros/sessionize.sql` open so Claude uses the real macro signatures, not hand-written UNNEST.
- The GA4 source schema (the `_ga4__sources.yml` column list from M1) - the authoritative list of raw columns staging may project.
- `CLAUDE.md` sec 5 - layer prefix `stg_ga4__`, materialization `view`, money `_in_usd` only, the `traffic_source` first-touch gotcha.

Files to provide

- `@docs/DBT_GUIDE.md` (sec 3, esp. 3.2/3.3/3.5) - the literal staging SQL + schema.yml.
- `@macros/get_event_param.sql` and `@macros/sessionize.sql` - the real macros M2 must call.
- `@models/staging/_ga4__sources.yml` - the `source('src_ga4','events')` target and the GA4 column contract.
- Use `/new-model staging stg_ga4__events` (the command built in M1) to scaffold, then fill from sec 3.2.

Prompts to use

Plan mode first:

```
[ROLE] You are implementing Helios M2, files models/staging/stg_ga4__events.sql,
models/staging/stg_ga4__event_params.sql, models/staging/stg_ga4__schema.yml. Follow
CLAUDE.md conventions (auto-loaded).
[SOURCE OF TRUTH] Copy/adapt the SQL VERBATIM from DBT_GUIDE.md sec 3.2, 3.3, 3.5
(@docs/DBT_GUIDE.md). These are pure projection views.
[UPSTREAM] Reference ONLY: the source via {{ source('src_ga4','events') }}
(@models/staging/_ga4__sources.yml) and the macros get_event_param / sessionize
(@macros/get_event_param.sql, @macros/sessionize.sql). Use ONLY raw GA4 columns that
```

```
exist in the attached source schema.
[TASK]
  - stg_ga4__events: parse_date('%Y%m%d', _table_suffix) AS event_date; explicit column
    list (NO SELECT *); ga_session_id / ga_session_number / page_location / session_engaged
    / engagement_time_msec / source / medium / campaign / gclid via get_event_param() with
    the correct types; traffic_source.{source,medium,name} as first_touch_* FALLBACK;
    device.* and geo.* light-flattened; ecommerce.*_in_usd money columns; session_key via
    sessionize() in the final CTE. Materialized: view.
  - stg_ga4__event_params: UNNEST(event_params) into the long (event x param key) table
    with string/int/float/double value slots + a coalesced value_string. This is the ONLY
    place UNNEST(event_params) appears. Materialized: view.
  - stg_ga4__schema.yml: the column docs + tests from sec 3.5 (session_key not_null where
    ga_session_id is not null; event_date/event_timestamp/user_pseudo_id not_null;
    param_key not_null; purchase_revenue_in_usd accepted_range min 0).
[CONSTRAINTS] NO joins, NO aggregations, NO SELECT * against the source, NO business logic
(that's intermediate/M3). Use ONLY canonical names. If a GA4 column you need is not in the
attached source schema, STOP and ASK.
[TEST-FIRST] Write/confirm stg_ga4__schema.yml tests, then run `dbt parse`, then
`dbt build --select staging` and show me the output (models built + tests passed).
[OUTPUT] Cite sec 3.2 / 3.3 / 3.5 per block.
```

If `dbt build --select staging` surfaces a real-data mismatch (e.g. a struct path differs on the sample), use:

```
The dbt build --select staging output shows <paste the exact BigQuery error>. Do NOT
guess a fix. Run `dbt show --select stg_ga4__events --limit 5` and
`{{ source('src_ga4','events') }}` schema via a one-off describe to find the real column
path, cite it, then correct ONLY that projection. Re-run `dbt build --select staging`.
```

Preventing hallucinations

- D3 (real columns) is the headline tactic for M2: staging is the layer that touches raw GA4, so a hallucinated column path is the prime failure. Pin the known-tricky paths: session id is `event_params.ga_session_id` via `get_event_param('ga_session_id','int')` (**not** a top-level `ga_session_id`); browser is `device.web_info.browser`; money is `ecommerce.purchase_revenue_in_usd` / `refund_value_in_usd` / `shipping_value_in_usd` / `tax_value_in_usd` (USD twins only - never the non-USD `purchase_revenue`).
- D6 - the machine is the catcher here, and it is decisive: `dbt build --select staging` actually runs the view against BigQuery. A hallucinated column (e.g. `device.browser` instead of `device.web_info.browser`, or a non-existent `ecommerce.gross_revenue`) fails with "Field not found" - you cannot fake-pass this. This is the staging-specific D6: compile is not enough, the build must touch the warehouse.
- D2/D5: enforce the "no business logic in staging" rule - if Claude adds a `WHERE event_name IN (...)` filter, a `GROUP BY`, or a join to the seed, reject it; that logic belongs to M3

intermediate. The only `WHERE` allowed is the `_TABLE_SUFFIX` / `var('ga4_start_date')` shard prune.
- Confirm `session_key` comes from `sessionize()` (the macro), not an inline `MD5( ... )` - a hand-inlined key is a D5 violation even if it's correct, because it duplicates the single source of truth.

Reviewing generated code

- R3: `dbt build --select staging` runs both views and their schema.yml tests - **show the pass**. This is the M2 acceptance gate.
- R2: no "Field not found" / "Unrecognized name" errors in the build log (proves no hallucinated columns - the staging-layer R6 equivalent, since views are dry-run-cheap and the build itself is the schema validation).
- Key uniqueness / not_null (the milestone-specific checks): `stg_ga4__events.session_key` not_null where `ga_session_id is not null` passes; `event_date`, `event_timestamp`, `user_pseudo_id` not_null pass; `stg_ga4__event_params.param_key` not_null passes. session_key is **not** unique at staging grain (one row per event, many events per session) - do not add a `unique` test here; uniqueness is asserted at the session grain in M3/M4.
- R5 (read the diff vs conventions): both models are `materialized: view`; explicit column lists (grep for `select *` against `{{ source(` - there must be none; the only `select *` is over the already-explicit CTE in `stg_ga4__events`'s `final`); `UNNEST(event_params)` appears in exactly one file (`stg_ga4__event_params`); `_in_usd` money columns only.
- R4: every projected column maps to a real GA4 field or a `get_event_param()` call; no invented metric/column names.
- R1: each SQL block cites sec 3.2 / 3.3; schema.yml cites sec 3.5. Then /clear before M3 (the sessionization keystone gets its own clean session and golden tests first).